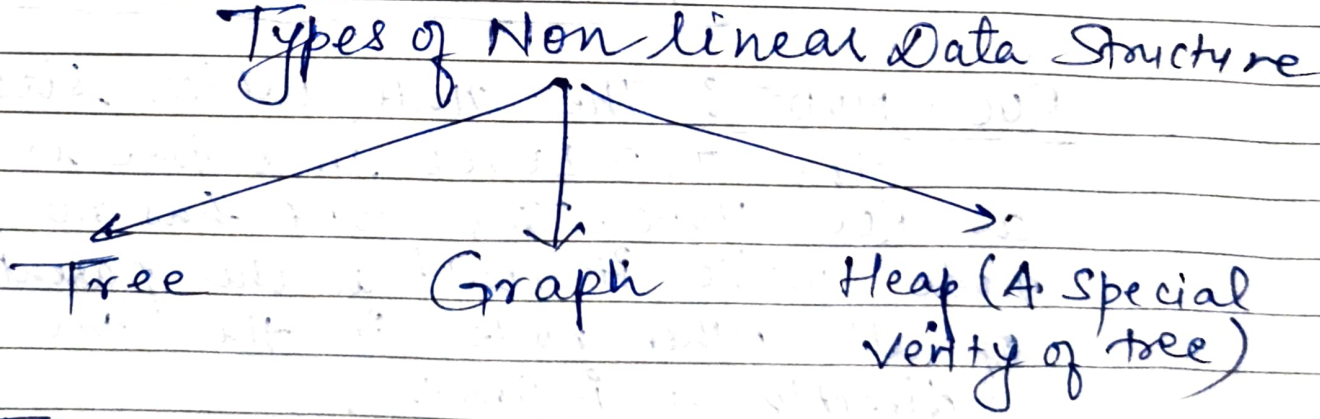
:- Non linear Data Structure :-

A non linear Data structure is a data structure where every element might be connected with more than one element after it.

Real world Example:

1. File System :- Where we have drives, each drive has multiple folders, each folder have multiple subfolders and further each of these subfolder might contain multiple files.

2. Any social networking plateform where we have multiple members and each of these members can have their own friend list.



Tree :→ A Tree is a non linear Data structure where elements are arranged in a HIERARCHICAL MANNER.

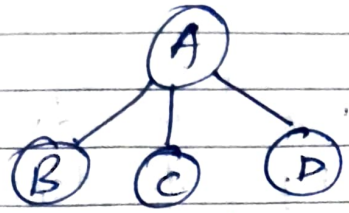
At every level in a tree we have elements which are called as nodes and below these elements we have another layer

of elements called as their child node.

BINARY TREE

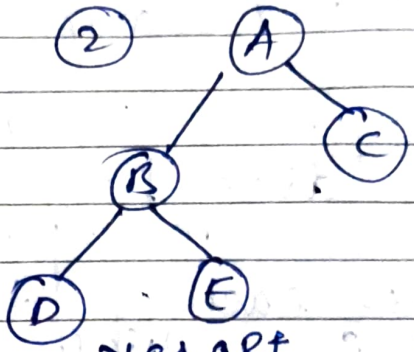
A Binary Tree is a special type of tree where every element at the most can have 2 children.

(1)



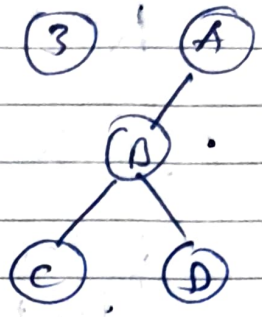
NOT a BT

(2)



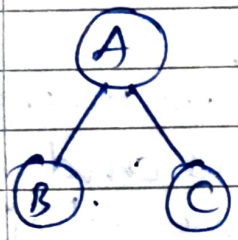
Yes. a BT

(3)



Yes a BT

(4)



Yes GBT

TERMINOLOGIES used with TREE

1. **ROOT NODE**: The node which sits at the TOP LEVEL in a tree is called as ROOT NODE. This NODE has no parent node while it itself is the parent node of all other nodes in the tree, either directly or indirectly.

2. **LEAF NODE**:- The Node which ~~dits~~ does not ~~any~~ has any child nodes is called a Leaf node. In our case the nodes D, E, F and G will be called as Leaf node.

3. NON-LEAF NODE :- The nodes which have atleast one child are called as NON-LEAF NODE. In our case nodes B & C will be called as Non leaf nodes

4. LEVEL OF A NODE :- Level of a node is defined as the length of the path from the root node to that particular node. Also remember that root is always considerd to be at level 0.

5. ANCESTORS : The nodes which appear before a perticular node, but on the same path are called as its ANCESTORS.

6 DESCENDENTS :→ The nodes which appear after a particular node, but on same path are called as its DESCENDENTS.

7 HEIGHT OF THE TREE :- The height of a tree is defined as the level of the last leaf node +1. In our example the last leaf node is at level 2, so the height will be 3.

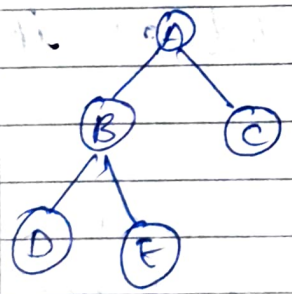
8. SIBLINGS :- Nodes which have the same parent will be called as SIBLINGS. In our Case B and Case C are Siblings, D & E are Siblings but E & F are not Siblings

9. FULL Binary TREE :- A Binary tree in which every node either has 2 children or 0 child will be called as a FULL BINARY TREE.

10. COMPLETE BINARY TREE :- A binary tree in which all the level except the last level are totally full and the nodes at the last level appear from left to right is called as Complete Binary tree.

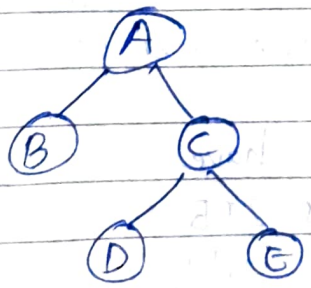
11. PERFECT BINARY TREE :- A Binary tree in which all the levels are totally full and all the leaf nodes are at the same level is called as a PERFECT BINARY TREE.

(1)



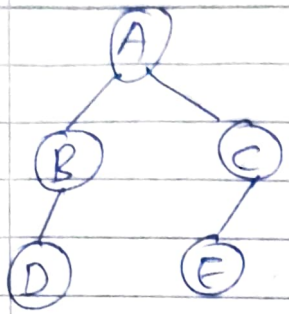
(1) Binary Tree ✓
(2) Complete Binary Tree ✓
(3) Perfect Binary Tree ✗
(4) FULL Binary Tree ✓

(2)



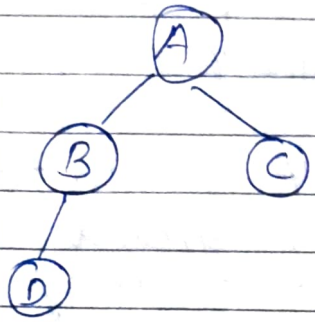
(1) Binary ✓
(2) Complete Binary ✗
(3) Perfect Binary ✗
(4) Full Binary ✓

(3)



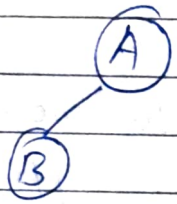
(1) Binary ✓
(2) Complete Binary ✗
(3) PERfect Binary ✗
(4) full Binary ✗

(4)



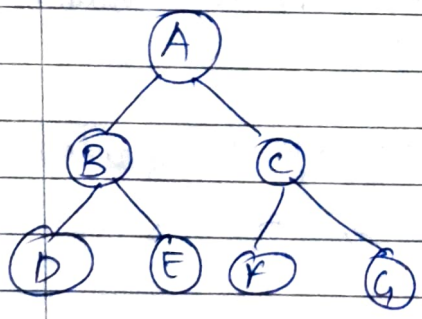
(1) Binary ✓
(2) Complete Binary ✓
(3) Perfect Bmary ✗
(4) Full Binary ✗

(5)



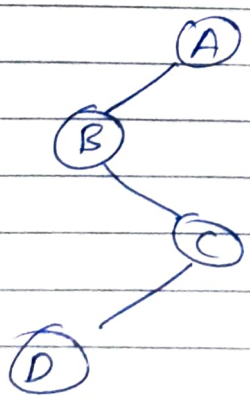
(1) Binary ✓
(2) Complete Binary ✓
(3) PERFECT Binary ✗
(4) FULL Binary ✗

(6)



(1) Binary ✓
(2) Complete Binary ✓
(3) PERFECT Binary ✓
(4) FULL BINARY ✓

(7)



(1) Binary ✓
(2) complete Binary ✗
(3) Perfect Binary ✗
(4) Full Binary ✗

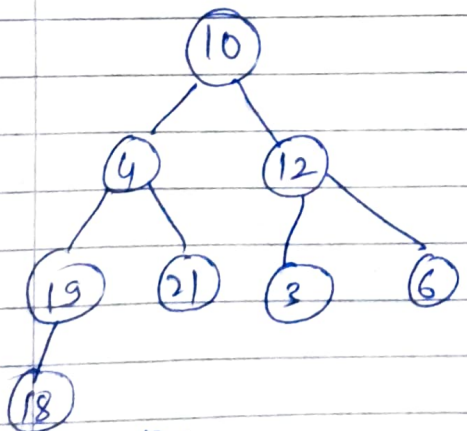
(8) (A)

1. Binary ✓
2. Complete Binary ✓
3. Perfect Binary
4. Full Binary ✓

12. BINARY SEARCH TREE :- A BST is a special type of Binary tree, where root and the child nodes have a specific relationship. The relationship is that, ~~either~~ the root node is always greater than its LEFT child and it is always smaller then its RIGHT CHILD.

OR we can say that all the nodes smaller then root node will always appear towards its left while all the nodes greater than root node will always appear towards right.

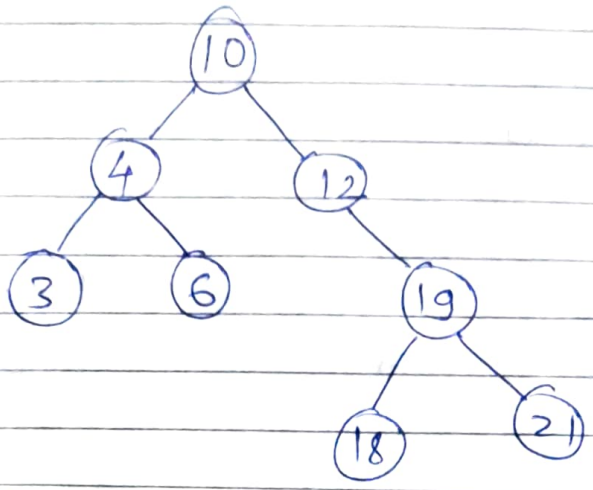
Draw a BINARY TREE of the following Data:
10, 4, 12, 19, 21, 3, 6, 18



Binary Tree

Draw a Binary Search Tree for the following data:

10, 4, 12, 19, 21, 3, 6, 18



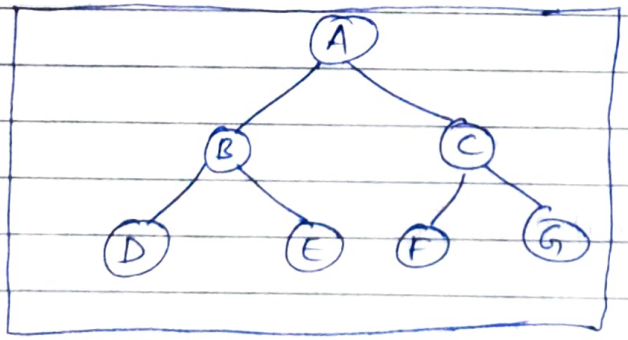
Binary Search TREE

REPRESENTING A BINARY TREE

Array

Linked list

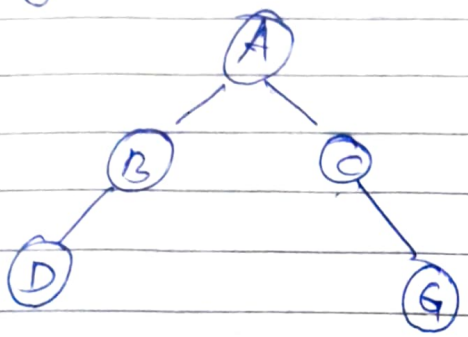
Representing A Binary Tree As an array



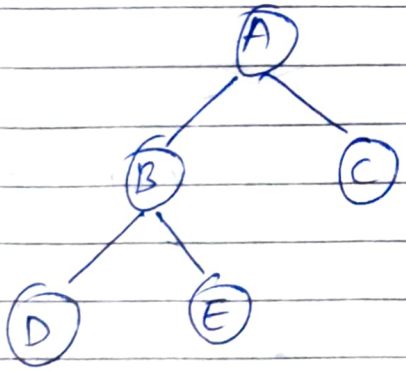
0	
1	A
2	B
3	C
4	D
5	E
6	F
7	G

FORMULLA for finding Parent & child Nodes Based upon Index:

1. Parent Node : index/2
2. Left child : index * 2
3. Right child : index * 2 + 1



0	
1	A
2	B
3	C
4	4
5	-
6	-
7	G



0	A
1	A
2	B
3	C
4	D
5	E

Important points With respect to Array Representation of Binary Tree

1. When we use Array representation of a Binary Tree Then we leave the 0th index blank.

2. Then we start with the root and we place it at the 1st index.

3 Further we move in the tree from left to right, filling each node at successive indexes in the tree, until we reach the last node.

4. We Skip the 0th index So that we can directly access parent node or left or right child of a node using the following formulas

a. PARENT : index/2
b. LEFT CHILD : index * 2
c. RIGHT CHILD : index * 2 + 1

where index is the index of the node in the array for which formula is being applied.

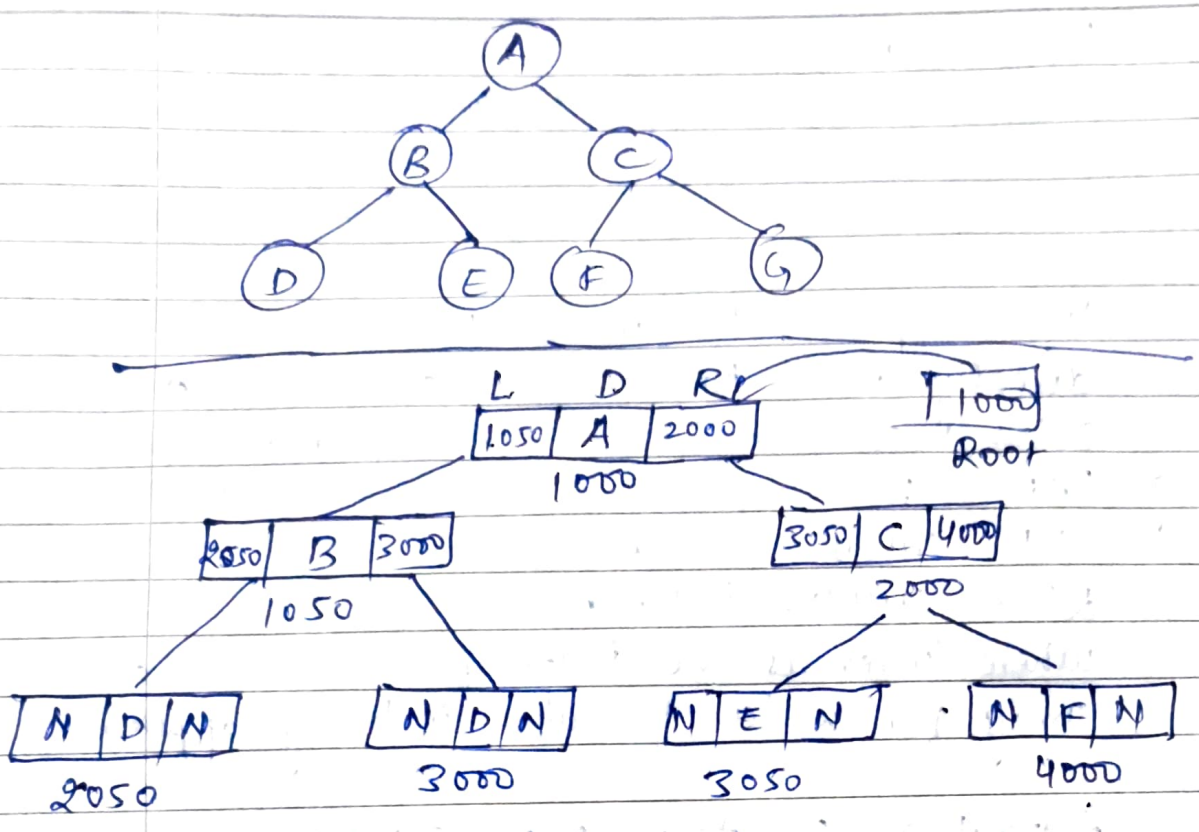
Advantage Of Array Representation

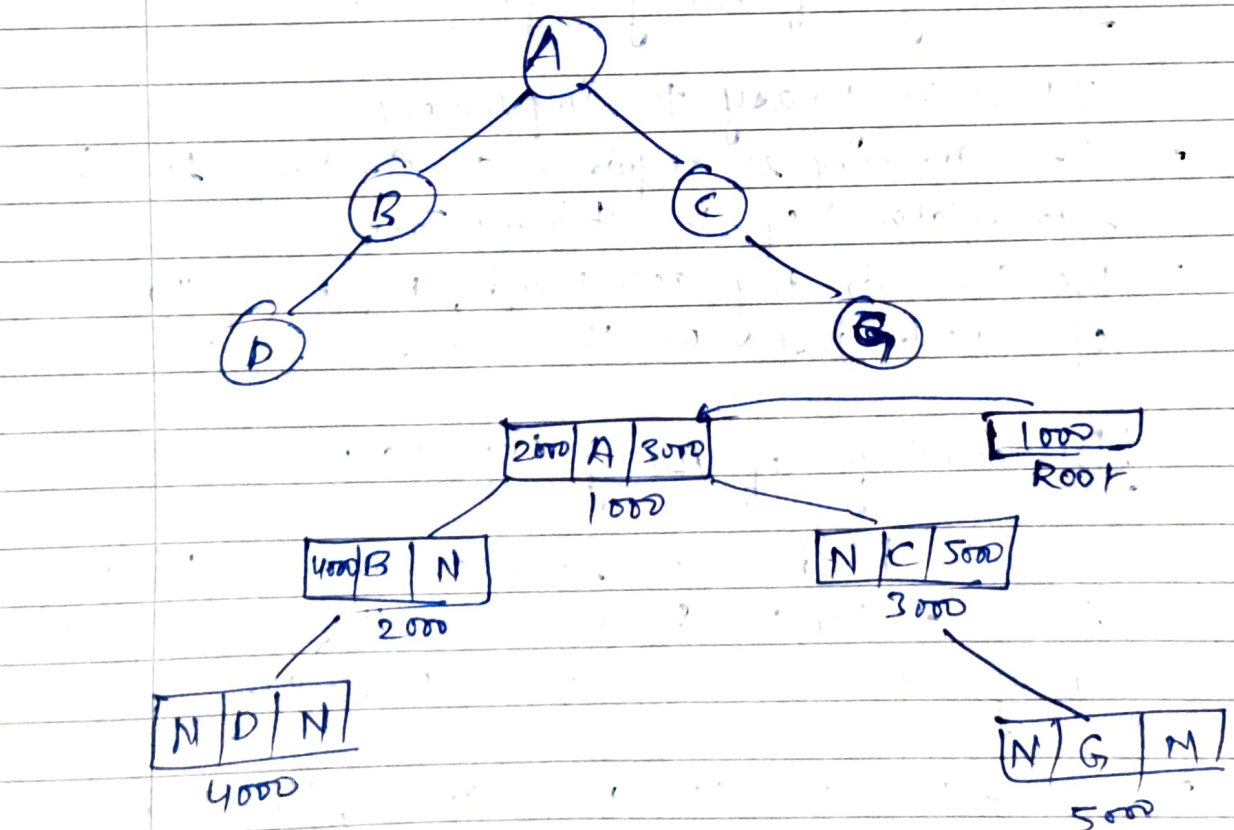
(1) It is very easy to implement
(2) Less memory is required as compared to linked representation of the Binary Tree.
(3) We can directly access left, Right or Parent of a node just on the basis of its index.

Disadvantage of Array Representation

(1) If the binary tree is not Complete Binary tree or Perfect Binary Tree then using array representation will waste Memory.

(2) Since we are using an array, The tree Can not dynamically grow or Shrink


A
B
C
D
E
F
G
L D R
1050 A 2000
1000
Root
1050 B 3000
1050
3050 C 4000
2000
N D N
2050
N D N
3000
N E N
3050
N F N
4000


A
B
C
D
G
2000 A 3000
1000
1000
Root
4000 B N
2000
N C 5000
3000
N D N
4000
N G M
5000

Important points about linked representation of a Binary Tree

1. We use a linked list (Kind/variety of doubly linked list) where each node has 3 members used for storing DATA and LEFT, RIGHT child pointers.

2. We also have a pointer to the ROOT NODE

3. The First element is designated as root node and all other nodes are added towards left or right of the root node. (Depending on the type of Binary Tree)

4. Moreover the LEFT & RIGHT pointers of every node will point to its left and right child and if the node has a missing left or Right child then these pointers will be set to NULL

Advantages of Linked Representation:-

1. It can be used to represent any kind of Binary tree i.e. BST, Full Binary Tree, Complete, Perfect etc.

2. No Need to create/allocate space for any node if it is not present in the tree.

3. The tree can easily grow and shrink dynamically.

Disadvantage of linked Representation

1. Require more memory as compared to an array Exactly 2n bytes per element EXTRA will be needed where n in the size of the pointer.
2. Require good knowledge of pointer to be implemented.
3. It is only possible to directly access the left and right child of a node For all other nodes traversal is required

Implementing a Binary Search Tree

```
Struct bst
{
  Struct bst *left;
  int data;
  Struct bst *right;
};
void append (struct bst **, int)
Void main()
{
  Struct bst *root = NULL;
  append (&root, 10);
  append (&root, 5);
  append (&root, 8);
}
```

```
Void append(Struct bst **pr, int x)
{
  Struct bst *p, *temp, *prev;
  p = (struct bst *) malloc(sizeof(Struct bst));
if (p == NULL)
  {
    Printf("Memory insufficient");
    return;
  } p -> data = x;
p -> left = p -> right = NULL;
if (*pr == NULL)
  {
    *pr = p;
    return;
  }
temp = *pr;
while (temp != NULL)
{
  Prev = temp;
  if (temp -> data > x)
      temp = temp -> left;
  else
      temp = temp -> right;
}
  if (Prev -> data > x)
          Prev -> left = p
  else
      Prev -> right = p
}
```

TREE TRAVERSAL ALGORITHM

Tree traversal visiting each and every node of the tree and accessing its data.

Although we can traverse in any way we like, but in DS we have 3 standard ways of tree traversal also called as Tree Traversal Algorithm. If we use these algorithms then there are many programming problems which we can easily solve using these algorithms.

These algorithms are:

1. PREORDER TRAVERSAL
2. INORDER TRAVERSAL
3. POSTORDER TRAVERSAL.

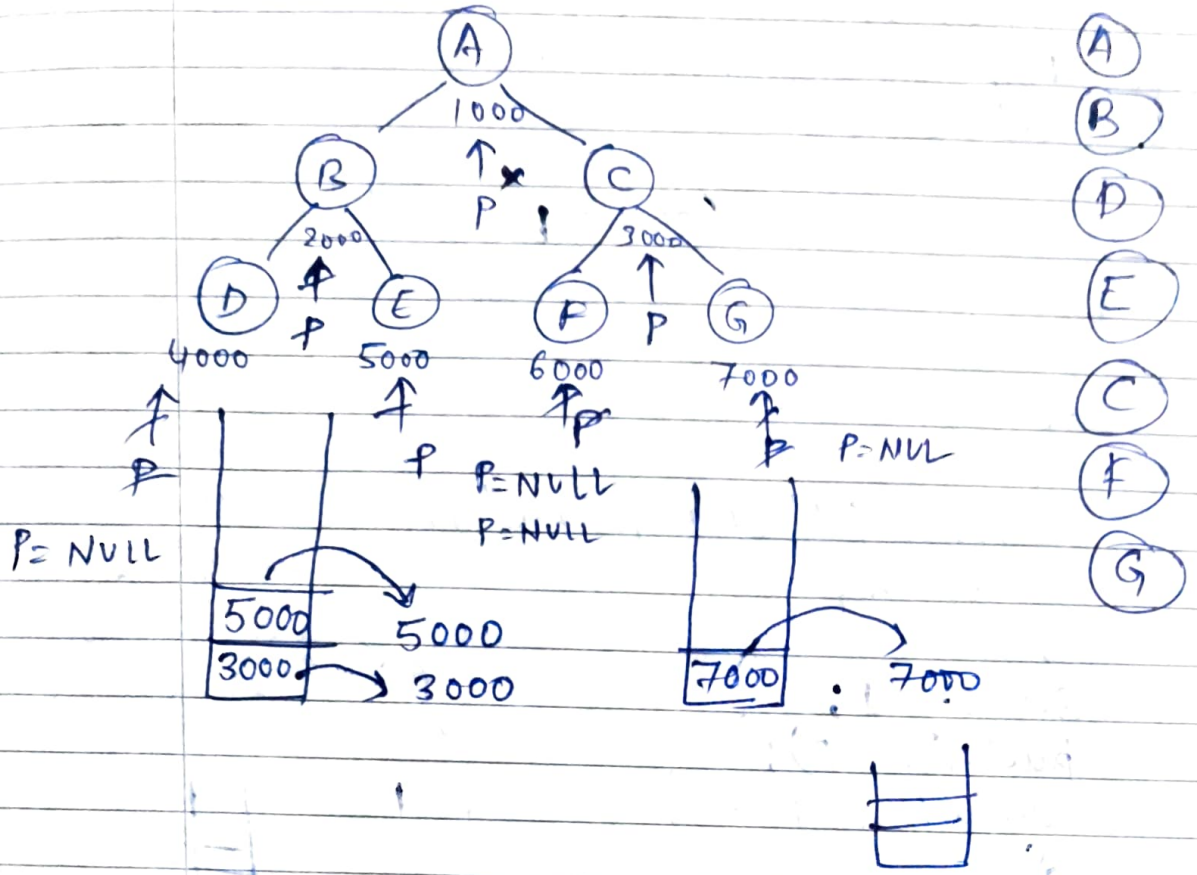
These names are based on the traversal patterns with respect to ROOT node.

A. In PREORDER traversal, we always get the ROOT node first, followed by LEFT child and then RIGHT child.

B. In INORDER traversal; we always get LEFT NODE First, followed by ROOT and then RIGHT child.

C. In POSTORDER TRAVERSAL, we always get the LEFT node first, followed by RIGHT child and then ROOT node.

PREORDER TRAVERSAL
(ROOT, LEFT, RIGHT)



Empty Stack
(Algo should stop)

Pseudocode for Preorder Traversal :-

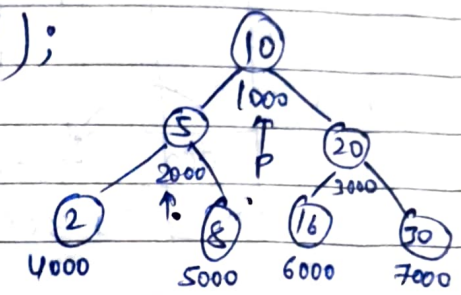
1. check whether the tree is empty or not.
2. If it is empty, then Print EMPTY TREE and return.
3. Proceed to the left most path from the Current Node and do the following:
   a. Print the Current node
   b. If the node has right child then PUSH it in the STACK.
   C. Move towards left
4. STOP when pointer becomes NULL
5. Check whether the stack is empty or Not.

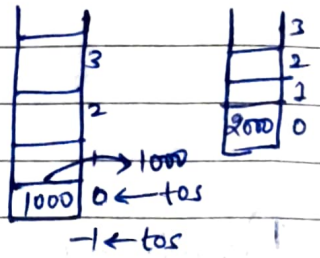
6. If the stack is empty then finish, Otherwise:

a. pop the top node.

b. Go to step 3.

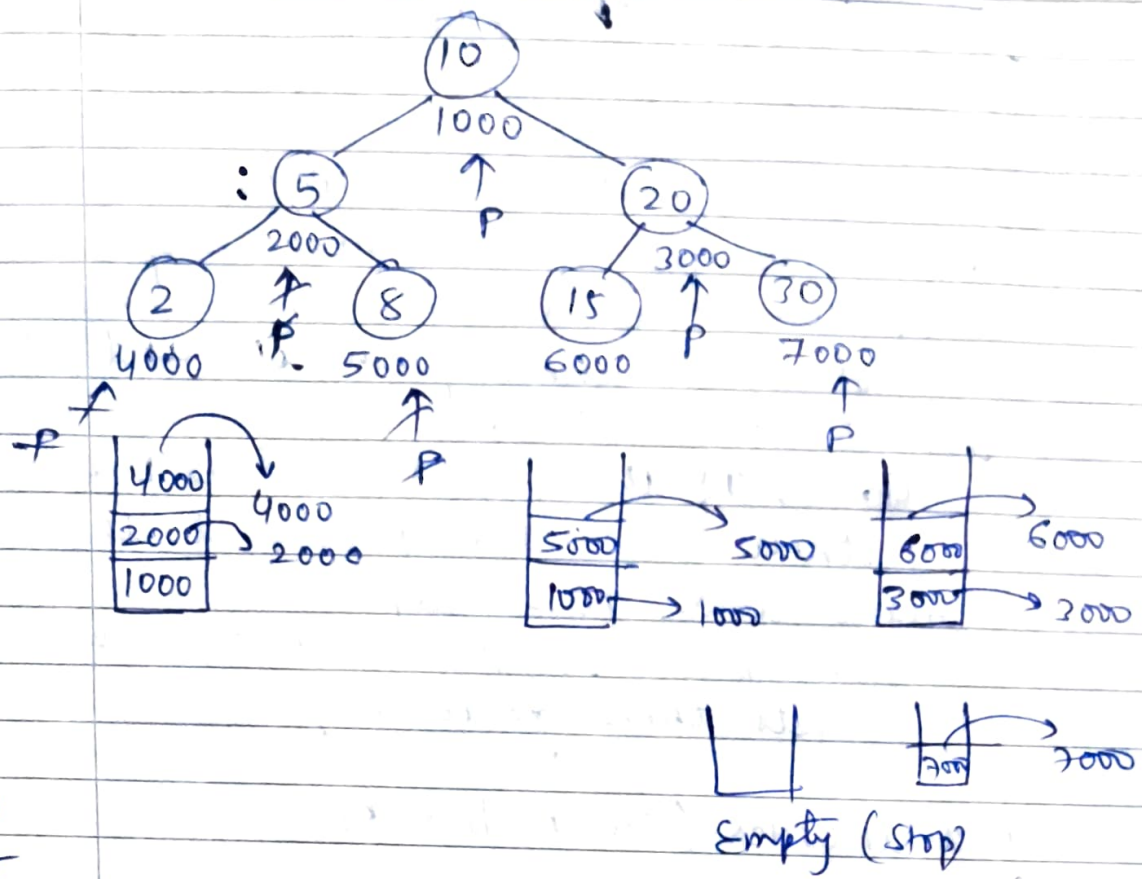
```
void preorder (struct bst *p)
{
  Struct Stack S;
  if(P==NULL)
  {
    Printf("Empty Tree");
    return;
  }
 S.tos=-1;
 push(&S, P);
  while(S.tos!=-1)
  {
    P= pop(&S);
   while(P!=NULL)
   {
    PrintF("%d\n", P→ data);
    if (P→right != NULL)
                Push(&S, P→right);
    P=P→ left;
   }
  }
}
```





10
5
2

INORDER TRAVERSAL.



2
5
8
10
15
20
30

If the binary tree is a BST, then the inorder traversal on this tree will always produce SORTED output.

Thus INORDER TRAVERSAL is also called as TREE SORT.

Pseudocode for INORDER TRAVERSAL:-

1. Check whether the tree is empty or not.
2. If it is empty then print EMPTY TREE and return.
3. Proceed down to the left most path from the current node and push each node in the stack.

4. Stop when the pointers becomes NULL.

5 Pop a node from the stack an do either of the following:

a. If we get NULL then stop

b. If we get a node from the stack then print it and check its RIGHT CHILD.

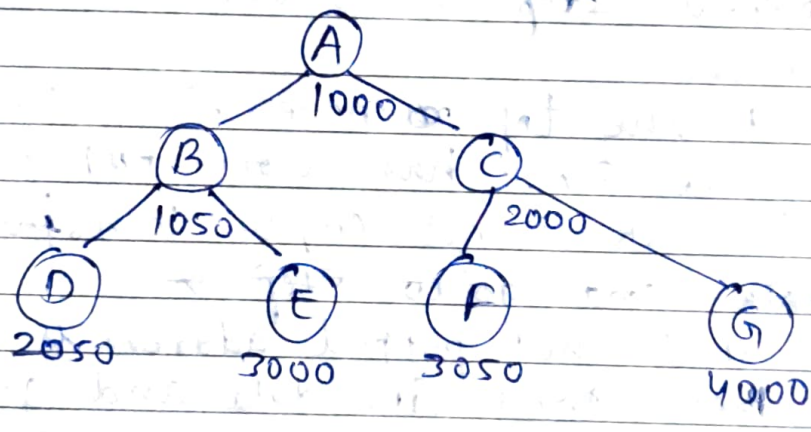
i. If the right child is present then move the pointer to the right and go to step 3.

ii. If the right child does not exists then go to step 5.

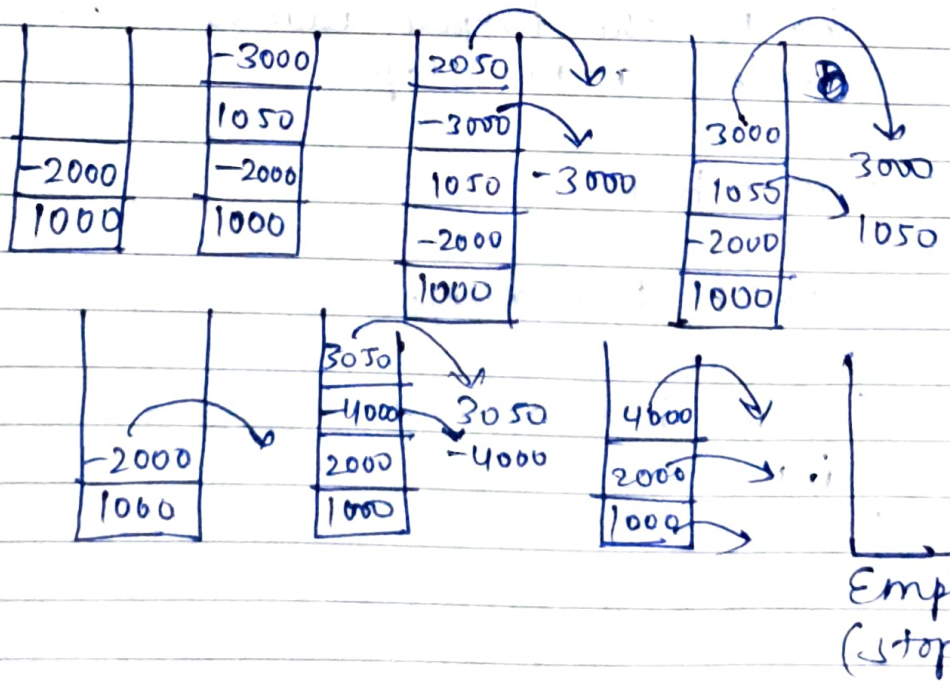
```
Void inorder (Struct bst *p)
{
   Struct Stack S;
   if (p == NULL)
   {
      PrintF ("Empty tree");
      return;
   }
S.tos = -1;
push_nodes:
            while (p != NULL)
            {
             Push (&S, p);
              p = p -> left;
            }
            P = pop(&s)
```

```
while (P != NULL)
{
  Printf ("\n%d", P -> data);
  if (P -> right != NULL)
  {
    P = P -> right;
    goto push_node;
  }
  P = Pop (&S)
}
}
```

POSTORDER TRAVERSAL



D
E
B
F
G
C
A



Pseudocode For Post Order ~~Solution~~ Traversal

1. Check whether the tree is empty or not
2. If it is empty, then print EMPTY TREE and return.
3. Push the Address of current Node in the stack by making it positive and check weather the node has a right child or not.
4. If it has a right child then push it in the STACK by making its address negative.
5. Move towards LEFT.
6. Repeat step 3 to 5 until pointer becomes NULL.
7. POP the top node from the Stack.
8. If the STACK has become empty then STOP.
9. If Stack is not empty then ~~print its data and go to step 7~~.
   a. If the popped address is positive then print its data and goto step 7.
   b. If the popped address is negative then go to step 3.

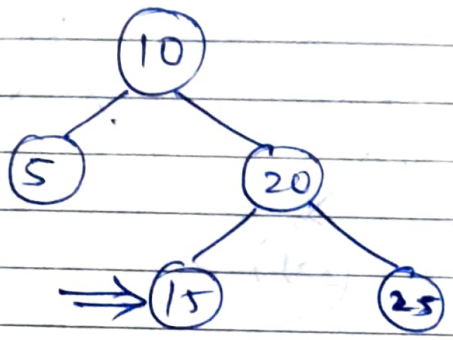
```
Struct bst
{
   Struct bst *left;
   int data;
   Struct bst *right
   int flag;
};
```

DELETION IN BINARY SEARCH TREE

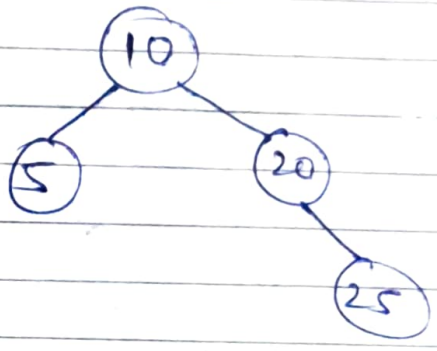
There are 3 possibilities :

1. The node to be deleted is LEAF NODE.
2. The node to be deleted has only 1 child (either left or right)
3. The node to be deleted has 2 children.

Case 1: Node to be deleted is LEAF NODE

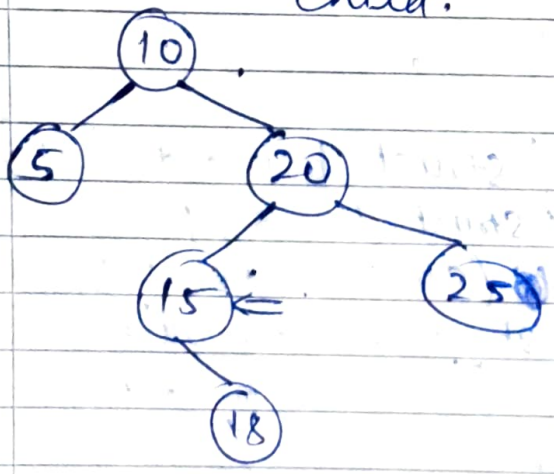


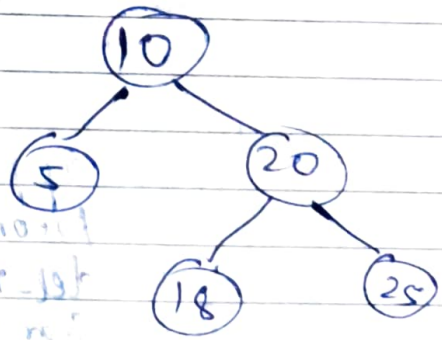
Before deletion



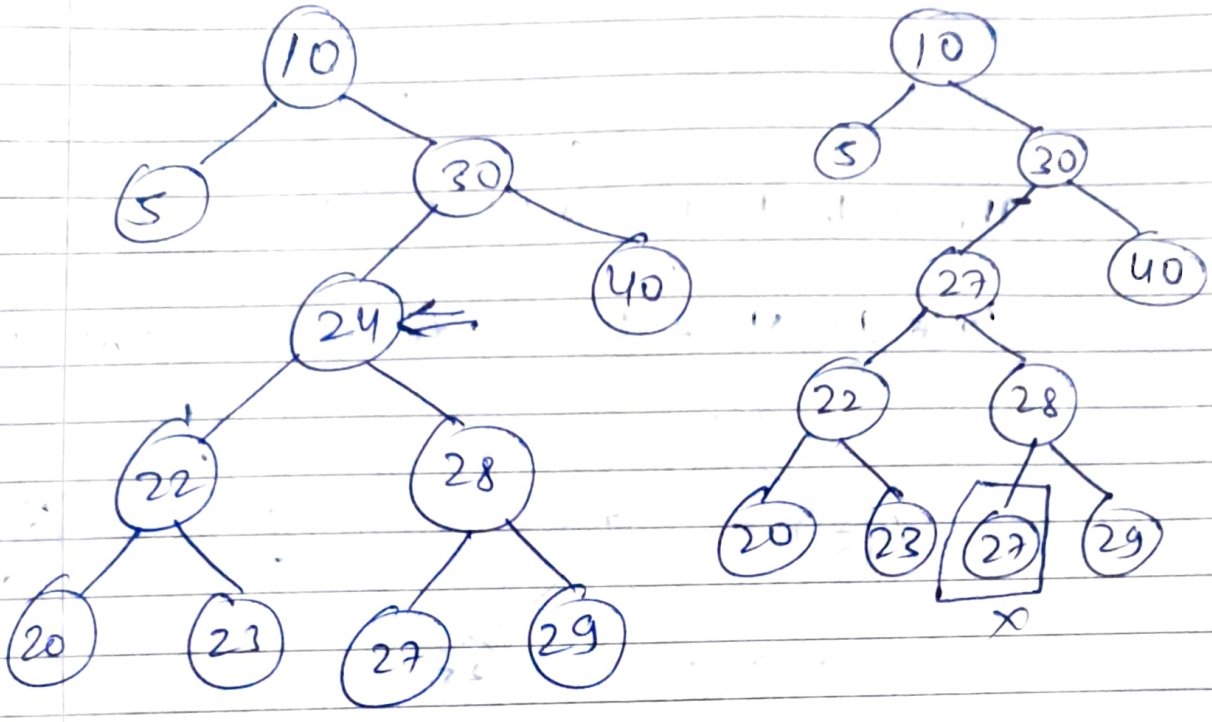
After deletion

CASE 2: Node to be deleted has 1 child.





CASE 3: Node to be deleted has 2 children




```
struct bst
{
  struct bst *left;
  int data;
  struct bst *right;
};
  void append(struct bst **, int);
  void preorder(struct bst *);
  void del_node(struct bst**, int);
  int search(struct bst *, int, struct bst**,
             struct bst**);
```

data to search

1 (found) 0 (not found)

root node address

pointer to pointer to parent node

Pointer to pointer child node

```
Void main()
{
  Struct bst * root = NULL;
   append (&root,10);
   append (&root, 20);
   append (&root, 7);
    :
    :
   del_node (&root, x);      Assume this Value is accepted from user as input
   Preorder (root);
   getch();
}

int Search (Struct bst *root, int x,
                Struct bst **pp,
                 Struct bst **pc)
{
  Struct bst *q = NULL;
  while (root != NULL)
  {
    if (root -> data == x)
      {
        *pc = root;
       *pp = q;
        Return 1;
      }
    q = root;
    if (root -> data > x)
            root = root -> left;
    else
          root = root -> right;
  }
  Return 0;
}
```

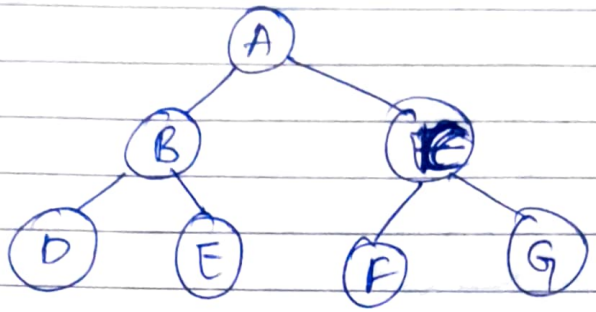
```
void del_node(struct bst **pr, int x)
{
  struct bst *child, *par, *q;
  int result;
  if (*pr == NULL)
  {
    Printf("Tree is Empty");
    return;
  }
result = search(*pr, &par, &child);
  if (result == 0)
  {
    Printf("Node Not found");
    return;
  }
if (child -> left != NULL && child -> right != NULL)
  {
    q = child -> right; Par = child;
    while(q -> left != NULL)
    { Par = q;
      q = q -> left;
    }
    child -> data = q -> data;
    child = q;
  }
  else if
 if (child -> left == NULL && child -> right
                    == NULL)
 {
   free(child);
    if (Par -> left == child)
                Par -> left = NULL;
```

```
        else
            Par -> right = NULL;
        free(child);
    }
    else if (child -> left != NULL && child->right
                                    == NULL)
    {
        if (Par -> left == child)
            Par -> left = child -> left;
        else
            Par -> right = child -> right;
        free(child);
    }
    else if (child -> right != NULL && child->left
                                    == NULL)
    {
        if (Par -> left == child)
            Par -> left = child -> right;
        else
            Par -> right = child -> right;
        free(child);
    }
}
```

(6) If it does not has any child then return

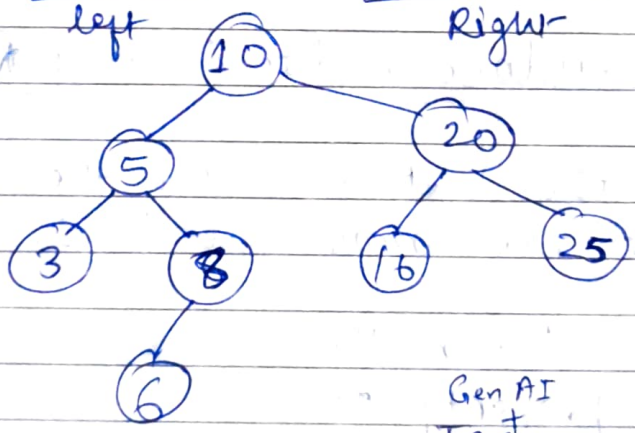
Constructing a Binary tree using Traversal order

Pre-order :- A, B, D, E, C, F, G (Root, Left, Right)
In-order :- D, B, E, A, F, C, G (Left, Root, Right)



(2) Pre : 10, 5, 3, 8, 6, 20, 16, 25
In : 3, 5, 6, 8, 10, 16, 20, 25

Left Right

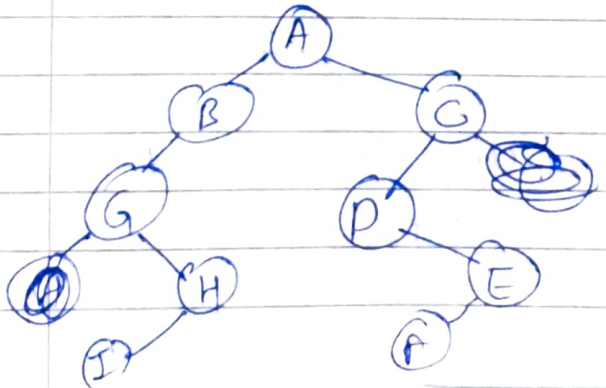


Gen AI
Isdc - 3.

(3) Pre: A, B, G, H, I, C, D, E, F
In: G, I, H, B, A, D, F, E, G

AI. - 7
TCS - 1

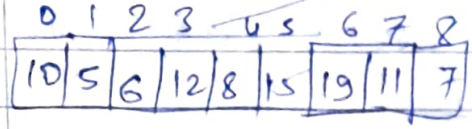
11 - 7 - 4

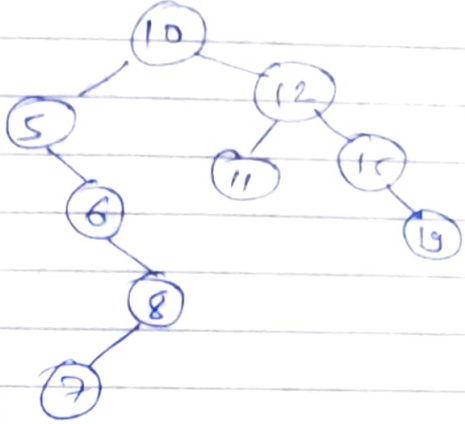


AVL TREE (Height balanced tree)

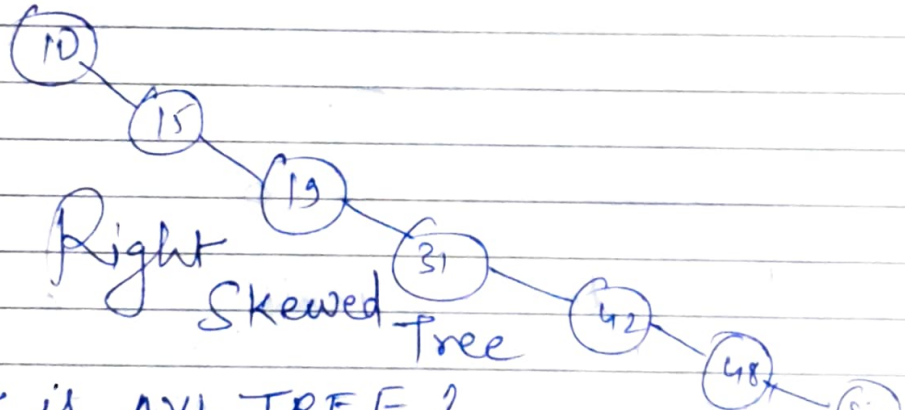
Why do we create a BST?
or
What is the benefit of a BST over an array?
We create a BST because compared to an array in a BST search time is less

10, 5, 6, 12, 8, 15, 19, 11, 7





10, 15, 19, 31, 42, 48, 51

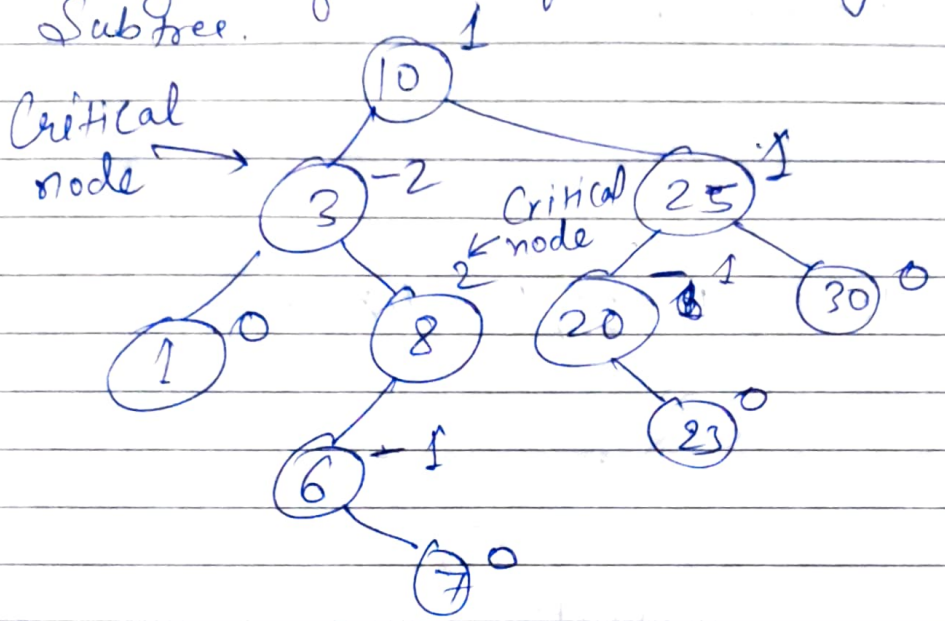


What is AVL TREE?
An AVL tree is a special type of binary search tree, where balance factor of each node of the tree falls b/w -1, 0, or +1. If the balance factor goes beyond this limit then the tree is considered to be UNBALANCED and node whose BALANCE FACTOR is beyond given the limit, is called a CRITICAL NODE. In this case before adding a

new node to the tree we first have to balance it using a technique called ROTATIONS

WHAT IS BALANCE FACTOR?

The Balance factor of a node is defined as the difference between the heights of its left and right Subtree.



How do we create an AVL TREE?
To create an AVL Tree we do following

1. We start the process of creating the tree just like a BST.
2. As Soon as we add a new node in the tree We check the BALANCE FACTOR of Each NODE.
3. If the BALANCE FACTOR OF Each node is within the range (-1, 0, +1), then We can proceed to add the next node.
4. But if the BALANCE FACTOR is not within the given range, then that node becomes

CRITICAL NODE and the tree becomes UNBALANCED.

5. In this case before we can add the next node, we first BALANCE THE TREE using ROTATIONS

How do we Balance a TREE?

To balance a tree we perform ROTATIONS

Types of ROTATIONS

1. Single Rotations
   a. LL Rotation
   b. RR Rotation
2. Double Rotation
   a. LR Rotation
   b. RL Rotation

Draw an AVL Tree of the following set of values:

10, 7, 3, 15, 19

